

МИНОБРНАУКИ РОССИИ
ФГАОУ ВО «МГТУ «СТАНКИН»
Кафедра Центр подготовки топ-специалистов в сфере ИТ ЛЕКЦИЯ №1

Введение в технологии разработки
программных средств:
от жёстких систем к эластичным экосистемам
Индустрии 5.0

по дисциплине
«Технологии разработки программных средств» Автор: Семенов А.С.

Москва, 2026 г.

Аннотация

В лекции представлен систематический обзор эволюции технологий разработки программного обеспечения — от первых машинных кодов до экосистем Индустрии 5.0. Вводятся базовые понятия: технология, инструментальное средство, парадигма программирования, модель жизненного цикла. Рассматривается сдвиг от централизованной, жёсткой разработки к распределённой, эластичной и человеко-центричной. Особое внимание уделяется цифровым двойникам как новой парадигме, их математическим основам и влиянию на все этапы разработки ПО. Анализируется концепция Индустрии 5.0, её отличие от Индустрии 4.0, и роль технологий в обеспечении устойчивости и гуманизации производства. Введён новый раздел, посвящённый анализу движущих сил сдвига парадигм, включая законы эволюции программного обеспечения М. Лемана [4, 5, 6] и концепцию смены парадигм Т. Куна [9]. Лекция содержит определения, сравнительные таблицы, иллюстрации и библиографию для углублённого изучения.

Ключевые слова

Технологии разработки ПО, инструментальные средства, жизненный цикл, цифровой двойник, эластичные системы, Индустрия 5.0, человеко-центричность, распределённая разработка, искусственный интеллект, MLOps, DevOps, законы эволюции ПО, сдвиг парадигм.

Содержание

Введение: базовые понятия	3
Технология разработки программных средств	3
Инструментальные средства (инструменты)	3
Парадигма программирования	3
Модель жизненного цикла (ЖЦ)	3
Движущие силы сдвига парадигм: что стоит за эволюцией?	3
Уровень 1: Внешние силы — требования меняющегося мира	3
Уровень 2: Внутренние силы — законы эволюции самого ПО	4
Уровень 3: Движущие силы-катализаторы — инструменты и абстракции	4
Сводная таблица: Три уровня движущих сил	5
Эволюция технологий: от машинных кодов к ИИ-агентам	6
Пласты сдвига парадигм: стек технологий	6
Сдвиг парадигм: от жёстких систем к эластичным экосистемам	7
Что такое «сдвиг парадигмы» в разработке ПО?	7
Сравнительная таблица: эпохи разработки	7
Что такое эластичные системы?	7
Цифровые двойники (Digital Twins): новая парадигма разработки	8
Определение и базовые понятия	8
Типы цифровых двойников	8
Как цифровые двойники меняют разработку ПО	8
Математические основы цифровых двойников	8
Моделирование из первых принципов (First Principles)	8
Моделирование на основе данных (Data-Driven)	8
Гибридное моделирование	8
Пример математической формализации	9
Индустрия 5.0: от эффективности к устойчивости и человеко-центричности	9
Что такое Индустрия 5.0?	9
Отличие Индустрии 4.0 от 5.0	9
Как технологии поддерживают Индустрию 5.0	10
Почему упор на цифровые двойники?	10
Наглядный пример: эволюция разработки системы управления роботом	10
Индустрия 3.0 (1970–1990): «Робот — это станок»	10
Индустрия 4.0 (2000–2020): «Робот — это источник данных»	11
Индустрия 5.0 (2020+): «Робот — это коллега»	11
Сводная таблица сравнения	12
Как нейросети делают разработку человеко-центричной?	12
Что это значит для разработчика?	12
Организация труда и инструменты в эпоху 5.0	13
Сдвиг в организации команд	13
Инструментальная поддержка новых ролей	13
Уровни инструментальной поддержки	13

Влияние цифровых двойников и эластичных систем на сокращение времени проекта	14
Факторы ускорения	14
Совокупный эффект	14
Заключение и связь с курсом	14

Введение: базовые понятия

Прежде чем перейти к историческому обзору и современным трендам, определим ключевые термины, которыми будем оперировать на протяжении всей лекции.

Технология разработки программных средств

Технология разработки программных средств (software development technology) — это совокупность методов, процессов, инструментов и организационных практик, используемых для создания, сопровождения и эволюции программных продуктов. Она охватывает весь жизненный цикл — от анализа требований до вывода системы из эксплуатации [1].

Инструментальные средства (инструменты)

Инструментальные средства (software tools) — это программы или программные комплексы, которые автоматизируют отдельные задачи в рамках технологии разработки. Они делятся на группы:

- **Инструменты анализа и проектирования** — CASE-средства, UML-редакторы.
- **Инструменты разработки** — IDE, компиляторы, отладчики.
- **Инструменты управления версиями** — Git, Subversion.
- **Инструменты тестирования** — JUnit, Selenium, тестовые фреймворки.
- **Инструменты сборки и развёртывания** — Maven, Gradle, Jenkins, Docker.
- **Инструменты управления проектами** — Jira, Trello, Asana.

Парадигма программирования

Парадигма программирования — это стиль мышления о программировании, совокупность концепций, определяющих структуру программ и способы их построения. Основные парадигмы: императивная (процедурная), объектно-ориентированная, функциональная, логическая, декларативная. Смена парадигм — один из ключевых сдвигов в истории разработки [2].

Модель жизненного цикла (ЖЦ)

Модель жизненного цикла ПО — это структура, описывающая последовательность этапов, через которые проходит программный продукт от зарождения идеи до снятия с поддержки. Модели делятся на последовательные (каскадная, V-модель), итеративные (инкрементная, спиральная) и адаптивные (Agile, DevOps). Выбор модели определяет организацию работы команды и инструментальную поддержку [3].

Движущие силы сдвига парадигм: что стоит за эволюцией?

Сдвиг парадигм в разработке ПО — не случайность и не чья-то прихоть. Это результат сложного взаимодействия трёх уровней сил, которые можно сравнить с эволюцией в биологии: выживают не самые сильные, а те, кто лучше адаптируется к постоянно меняющейся среде [9].

Уровень 1: Внешние силы — требования меняющегося мира

Это главный «двигатель», задающий направление всему развитию.

- **Экономическая эффективность и скорость (Time-to-Market)**. Бизнесу нужно быстрее и дешевле доставлять ценность пользователю. Это привело к появлению **Agile**, а затем и **DevOps**, который автоматизирует всё, что можно автоматизировать, чтобы ускорить поставку.

- **Рост сложности.** Мир стал цифровым. Мы перешли от решения изолированных задач к созданию глобальных, распределённых систем, которые должны работать в режиме 24/7 и обслуживать миллионы пользователей. Старые монолитные подходы просто не справляются, что привело к появлению **микросервисов** и **облачных технологий**.
- **Переход к Индустрии 5.0.** Это новый вектор развития, предложенный Европейской комиссией [8]. Главная цель больше не просто «эффективность любой ценой». На сцену выходят **человекоцентричность, устойчивость и жизнестойкость (resilience)**. Технологии должны подстраиваться под человека, а не наоборот. Именно эта сила выводит на первый план **цифровые двойники** [7].
- **Внешние шоки и регуляторное давление.** Пандемии, сбои в цепочках поставок, новые законы о данных и устойчивом развитии заставляют компании делать системы более надёжными, безопасными и прозрачными.

Уровень 2: Внутренние силы — законы эволюции самого ПО

Эти силы действуют внутри самой инженерии и были описаны ещё в 1970-х годах Меиром Леманом [4]. Его знаменитые **законы эволюции программного обеспечения** объясняют, почему разработка — это вечный процесс изменений [5, 6].

Основные законы Лемана:

1. **Непрерывное изменение (Continuing Change):** Любая используемая в реальном мире программа должна постоянно адаптироваться, иначе она становится бесполезной.
2. **Увеличение сложности (Increasing Complexity):** По мере развития программы её сложность неизбежно растёт. Именно это давление заставляет нас придумывать новые абстракции, инструменты и архитектуры, чтобы эту сложность обуздать.
3. **Саморегулирование (Self-Regulation):** Процесс эволюции программы является саморегулируемым, с близким к нормальному распределением.
4. **Сохранение организационной стабильности (Conservation of Organisational Stability).**
5. **Сохранение знакомства (Conservation of Familiarity).**
6. **Непрерывный рост (Continuing Growth).**
7. **Снижение качества (Declining Quality).**
8. **Система обратной связи (Feedback System).**

Как отмечается в исследованиях [5], «каждый закон encapsulate организационные и другие факторы, лежащие вне области программной инженерии и вне компетенции разработчиков ПО». Это означает, что движущие силы сдвига лежат не только в технической, но и в организационной, экономической и социальной плоскостях.

Уровень 3: Движущие силы-катализаторы — инструменты и абстракции

Это те самые «рычаги», с помощью которых мы отвечаем на вызовы внешнего мира. Они не просто следуют за изменениями, а сами становятся их движущей силой.

- **Рост уровня абстракции.** *Это, пожалуй, главный двигатель всей истории программирования.* Мы постоянно поднимаемся на новый уровень, отходя от «как» к «что». Мы перешли от машинных кодов к ассемблеру, от ассемблера к языкам высокого уровня, от них к объектно-ориентированному

и функциональному программированию. Сейчас мы стоим на пороге **пятого поколения**, где абстракцией становится сама **спецификация**, а код генерируется из неё с помощью ИИ. Этот рост абстракции и есть та самая сила, которая позволила перейти от жёстких систем к эластичным и интеллектуальным.

- **Искусственный интеллект (ИИ).** ИИ стал ключевым катализатором. Он позволяет автоматизировать не только рутинные задачи, но и целые этапы создания ПО, делая возможным переход на новый уровень абстракции. Многие исследователи считают генеративный ИИ «самой disruptive инновацией в программной инженерии со времён появления Интернета». Роль разработчиков смещается от реализации к оркестрации, фокусируясь на решении проблем и системном дизайне.
- **«Болезненный» опыт и ошибки.** Кризисы и провалы проектов — тоже мощные двигатели. Именно «кризис программного обеспечения» 1960–70-х породил **инженерию ПО** как дисциплину. Как отмечается в исследованиях [2], «кризис ПО был парадоксально порождён успехом аппаратного обеспечения, а не его провалом». Неудачи с монолитами привели к успеху **микросервисов**. Каждый раз, наступая на грабли, мы изобретаем что-то новое.

Сводная таблица: Три уровня движущих сил

Таблица 1: Три уровня движущих сил сдвига парадигм

Уровень силы	Что это?	К чему привело?	Примеры и источники
1. Внешние (Мир)	Экономика, сложность, новые социальные требования (Индустрия 5.0)	Появление Agile, DevOps, микросервисов, фокус на человеке и устойчивости	Требование быстрой доставки функций; создание глобальных распределённых систем; концепция Индустрии 5.0 [8]
2. Внутренние (Законы эволюции ПО)	Непрерывное изменение, рост сложности (законы Лемана)	Постоянное давление, требующее новых инструментов и методов для управления сложностью	Законы Лемана [4, 5]; каждый закон encapsulate организационные факторы
3. Катализаторы (Инструменты)	Рост уровня абстракции, ИИ, «болезненный» опыт	Реализация ответов на внешние вызовы. Смена парадигм программирования	Переход от машинных кодов к языкам ВУ; генеративный ИИ как самая disruptive инновация со времён Интернета; кризис ПО 1960-70-х как триггер появления инженерии ПО [2]

Итог: движущей силой является не какая-то одна причина, а **постоянное давление** со стороны быстро меняющегося мира, усугубляемое **внутренними законами роста сложности** самого ПО. Мы отвечаем на это давление, создавая **новые уровни абстракции** и используя новые технологии, такие как **ИИ**, которые, в свою очередь, становятся новыми драйверами для следующего витка эволюции. Это бесконечный, самоускоряющийся процесс [9].

Эволюция технологий: от машинных кодов к ИИ-агентам

Ниже представлена временная ось, показывающая эволюцию парадигм программирования, моделей ЖЦ и ключевых инструментов. Все подписи — многострочные, чтобы избежать наложения.

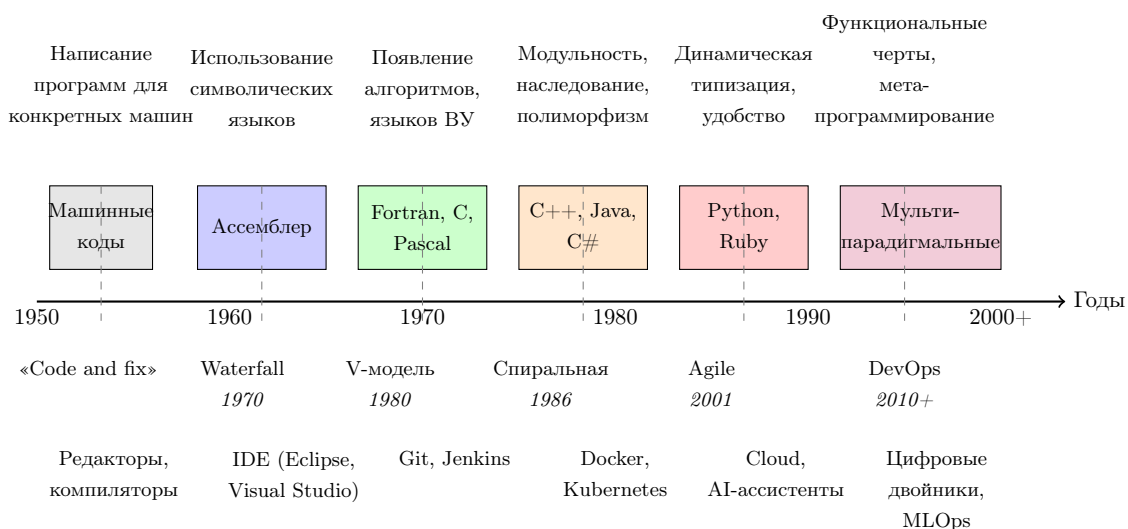
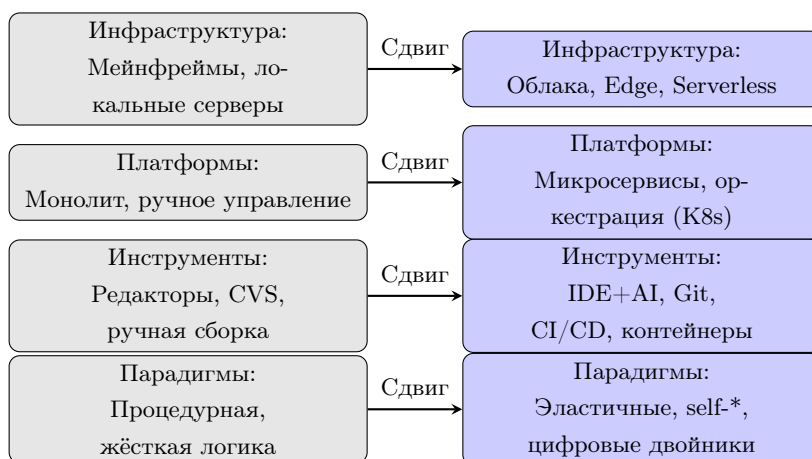


Рис. 1: Интегрированная временная ось: языки, модели ЖЦ и инструменты

Пласты сдвига парадигм: стек технологий

Чтобы понять глубину изменений, представим разработку ПО как четырёхуровневый стек, где каждый уровень эволюционирует по-своему.

Пласты сдвига парадигм в разработке ПО



Каждый уровень эволюционирует от жёстких, централизованных решений к распределённым, эластичным и интеллектуальным.

Рис. 2: Пласты сдвига парадигм и стек технологий

Сдвиг парадигм: от жёстких систем к эластичным экосистемам

Что такое «сдвиг парадигмы» в разработке ПО?

Сдвиг парадигмы — это фундаментальное изменение в способе мышления и подходах к решению задач [9]. В истории программирования таких сдвигов было несколько:

- Переход от машинных кодов к языкам высокого уровня (1960-е).
- Переход от процедурного программирования к объектно-ориентированному (1980-е).
- Переход от монолитных архитектур к микросервисам (2010-е).
- Переход от статичных систем к эластичным, самоадаптируемым экосистемам (2020-е).

Сравнительная таблица: эпохи разработки

Таблица 2: Сдвиг парадигм в разработке ПО						
Критерий	Индустрия (1970–1990)	3.0	Индустрия (2000–2020)	4.0	Индустрия (2020+)	5.0
Цель	Автоматизация		Цифровизация, эффективность	эф-	Устойчивость, человеко-центричность [8]	
Архитектура	Монолит		Микросервисы		Эластичные, самоадаптирующиеся системы	
Подход к данным	Статические структуры		Большие данные, потоковая обработка		Цифровые двойники, непрерывный поток [7]	
Управление	Централизованное		Распределённое, жёсткое	но	Децентрализованное, автономное	
Роль человека	Оператор		Пользователь		Соавтор, партнёр	
Инструменты	Компиляторы, тер-миналы		IDE, Git, CI/CD		AI-агенты, цифровые двойники, Low-Code	
ЖЦ	Водопад		Agile, DevOps		DevOps, непрерывная эволюция	

Что такое эластичные системы?

Эластичные системы (elastic systems) — это системы, способные автоматически изменять свою структуру, поведение и потребление ресурсов в ответ на изменение внешних условий, нагрузки или целей. Они обладают свойствами:

- **Self-Healing** — самовосстановление после сбоев.
- **Self-Optimizing** — самооптимизация под текущую нагрузку.
- **Self-Adapting** — адаптация архитектуры к новым требованиям.

Примером являются **эмерджентные микросервисы** (emergent microservices), которые самостоятельно образуются и перестраиваются для решения возникающих задач.

Цифровые двойники (Digital Twins): новая парадигма разработки

Определение и базовые понятия

Цифровой двойник (Digital Twin, DT) — это динамическая виртуальная модель физического объекта, процесса или системы, которая непрерывно синхронизируется с реальным прототипом через поток данных и позволяет моделировать, прогнозировать и оптимизировать его поведение [7]. Ключевое отличие от обычной модели — **двунаправленная связь**: двойник не только отражает реальность, но и может воздействовать на неё.

Типы цифровых двойников

- **Digital Twin Prototype (DTP)** — создаётся до физического объекта для тестирования концепций.
- **Digital Twin Instance (DTI)** — привязан к конкретному экземпляру объекта (например, конкретному двигателю).
- **Digital Twin Aggregate (DTA)** — объединяет данные от множества экземпляров для анализа совокупности.

Как цифровые двойники меняют разработку ПО

Цифровые двойники не являются просто ещё одним инструментом — они представляют собой **новую парадигму**, влияющую на каждый этап ЖЦ:

- **Требования:** двойник позволяет проигрывать сценарии и проверять требования до начала разработки.
- **Проектирование:** архитектура тестируется на двойнике системы, выявляются узкие места.
- **Разработка:** двойник служит «песочницей» для тестирования кода в условиях, приближенных к реальным.
- **Тестирование:** комплексная валидация поведения ПО до развёртывания.
- **Эксплуатация:** непрерывный мониторинг, прогнозирование отказов, self-healing.
- **DevOps:** двойник помогает преодолевать вызовы, повышая надёжность и масштабируемость.

Математические основы цифровых двойников

За каждым цифровым двойником стоит серьёзный математический аппарат. Выделяют три основных подхода:

Моделирование из первых принципов (First Principles)

Построение физико-математических моделей на основе законов физики, химии, механики. Обеспечивает высокую точность, но требует глубокого понимания предметной области.

Моделирование на основе данных (Data-Driven)

Использование методов машинного обучения (нейронные сети, полиномиальная регрессия) для построения модели по историческим данным. Хорошо работает там, где физика сложна или неизвестна.

Гибридное моделирование

Сочетание первых двух подходов, где физическая модель «дообучается» на данных.

Пример математической формализации

Обобщённая модель цифрового двойника может быть записана как:

$$y_k = F_{p_k}(\tilde{x}_k, \Psi_{p_k}(y_{k-1}, z_k))$$

где:

- y_k — состояние двойника в момент k ,
- $\tilde{x}_k$ — входные данные (наблюдения),
- z_k — управляющие воздействия,
- F и Ψ — операторы модели (могут быть как физическими уравнениями, так и нейросетями).

Такая формализация объединяет традиционные модели и методы машинного обучения в единую структуру.

Индустрия 5.0: от эффективности к устойчивости и человеко-центричности

Что такое Индустрия 5.0?

Индустрия 5.0 — это концепция, предложенная Европейской комиссией, которая дополняет Индустрию 4.0 [8]. Она ставит во главу угла не только эффективность, но и:

- **Человеко-центричность (Human-centric):** технологии подстраиваются под потребности человека, а не наоборот.
- **Устойчивость (Sustainability):** минимизация экологического следа, экономия ресурсов.
- **Жизнестойкость (Resilience):** способность систем выдерживать потрясения (пандемии, сбои цепочек поставок).

Отличие Индустрии 4.0 от 5.0

Таблица 3: Сравнение Индустрии 4.0 и 5.0

Критерий	Индустрия 4.0	Индустрия 5.0
Цель	Эффективность, оптимизация	Устойчивость, благополучие человека [8]
Роль человека	Оператор, контролёр	Соавтор, партнёр, центр системы
Технологии	IoT, большие данные, AI	Цифровые двойники, когнитивные системы, коллаборативные роботы [7]
Архитектура	Централизованные платформы	Децентрализованные, автономные агенты
Подход к данным	Сбор и анализ	Непрерывное моделирование и прогнозирование
Примеры	Умные заводы	Индивидуализированное производство, системы поддержки человека

Как технологии поддерживают Индустрию 5.0

- **Цифровые двойники** — позволяют моделировать влияние решений на человека и экологию, обеспечивают жизнестойкость.
- **Когнитивные системы** — анализируют состояние человека, адаптируют интерфейсы.
- **Коллаборативные роботы** — работают плечом к плечу с человеком, а не вместо него.
- **Эластичные системы** — автоматически перестраиваются под изменяющиеся условия, минимизируя стресс для человека.
- **MLOps** — управление жизненным циклом моделей AI, обеспечивающее их непрерывную актуализацию.

Почему упор на цифровые двойники?

Цифровые двойники являются **ключевым энблером** Индустрии 5.0, потому что:

1. Они обеспечивают **непрерывную эволюцию** систем, что соответствует требованию устойчивости.
2. Они позволяют **тестировать человеко-центричные сценарии** без риска для людей (например, моделирование воздействия на здоровье оператора).
3. Они дают возможность **прогнозировать** и предотвращать сбои, повышая жизнестойкость.
4. Они служат **платформой для интеграции AI**, превращая данные в осмысленные действия.
5. Они **соединяют физический и цифровой миры**, создавая единую информационную среду.

Наглядный пример: эволюция разработки системы управления роботом

Чтобы конкретно ощутить разницу между Индустрией 3.0, 4.0 и 5.0, рассмотрим, как менялась разработка ПО для управления промышленным роботом на протяжении этих эпох.

Индустрия 3.0 (1970–1990): «Робот — это станок»

Задача: Запрограммировать робота на выполнение строго определённой последовательности движений.

Как разрабатывалось ПО:

- Инженер писал код на низкоуровневых языках (часто — на языках программируемых логических контроллеров, PLC) **один раз**.
- ПО было жёстко привязано к конкретной модели робота и конкретной операции.
- Процесс разработки был **линейным и долгим**: спецификация → кодирование → тестирование на реальном оборудовании.

Пример из реальной практики: в прошлом коммерческие роботы выполняли простые повторяющиеся задачи, и любое изменение требований означало переписывание кода и перепрошивку контроллера.

Что чувствовал разработчик:

- Работал с «железом» на уровне электрических схем и таймингов.
- Любое изменение требований означало переписывание кода и перепрошивку контроллера.
- Общение с роботом — через пульт и светодиодные индикаторы.

Ключевые технологии: PLC, релейная логика, ручное программирование движений.

Индустрия 4.0 (2000–2020): «Робот — это источник данных»

Задача: Сделать робота «умным» — подключить его к сети, собирать данные о его работе, оптимизировать процессы.

Как разрабатывалось ПО:

- Появились **микросервисные архитектуры**: один сервис отвечает за движение, другой — за сбор данных, третий — за диагностику.
- Разработка ведётся **итеративно**, код хранится в Git, используются контейнеры (Docker) и облачные платформы.
- Внедряются **цифровые двойники** для виртуального тестирования.

Пример из практики: Индустрия 4.0 обеспечила технологическую основу для умных фабрик с высокой связностью и низким временем выполнения заказов.

Что чувствовал разработчик:

- Пишет код на Python или C++, работает с REST API, базами данных, очередями сообщений.
- Взаимодействует с дашбордами и облачными сервисами.
- Данные — в центре внимания: нужно собирать, анализировать, визуализировать.

Ключевые технологии: IoT, облачные вычисления, цифровые двойники, микросервисы, Git, CI/CD.

Индустрия 5.0 (2020+): «Робот — это коллега»

Задача: Сделать робота **партнёром**, который адаптируется к состоянию и потребностям человека.

Как разрабатывается ПО:

- Вместо жёстко заданных алгоритмов используются **нейросети и LLM**, которые позволяют роботу понимать **естественный язык и контекст**.
- Разработка ведётся с акцентом на **пользовательский опыт (UX)**: интерфейсы должны быть интуитивными, снижать когнитивную нагрузку.
- Внедряются **человеко-центричные цифровые двойники**, которые учитывают не только технические параметры, но и усталость, позу, даже эмоциональное состояние оператора.

Пример из практики: в современных исследованиях предлагаются LLM-фреймворки для планирования задач робота на естественном языке, чтобы улучшить коммуникацию между человеком и роботом, одновременно снижая умственную нагрузку на оператора. Модуль восприятия использует компьютерное зрение для отслеживания движений рук и классификацию жестов.

Что чувствует разработчик:

- Работает с **модальными интерфейсами**: голос, жесты, взгляд (например, используя MediaPipe для распознавания жестов).
- Интегрирует **LLM**, чтобы оператор мог сказать роботу: «Собери эту деталь, но чуть медленнее, я устал», и робот поймёт и адаптируется.
- Проектирует **адаптивные интерфейсы**, которые подстраиваются под уровень подготовки оператора.

Ключевые технологии: Нейросети, LLM, компьютерное зрение, XR/VR, человеко-центричные цифровые двойники.

Сводная таблица сравнения

Таблица 4: Сравнение трёх эпох на примере разработки ПО для робота

Аспект	Индустрия 3.0	Индустрия 4.0	Индустрия 5.0
Цель ПО	Выполнить операцию	Собрать и обработать данные	Улучшить работу человека
Архитектура	Монолит, жёсткая логика	Микросервисы, контейнеры	Гибридные, самоадаптирующиеся системы
Интерфейс	Пульт, светодиоды	Дашборды, веб-интерфейсы	Голос, жесты, VR/AR
ИИ	Отсутствует	Для анализа данных	Для адаптации к человеку
Роль разработчика	Программист «железа»	Full-stack инженер	Оркестратор AI-агентов
Главная метрика	Скорость/точность	Эффективность/данные	Удовлетворённость и безопасность человека

Как нейросети делают разработку человеко-центричной?

Использование нейросетей — это не просто «ещё одна технология». Это **инструмент**, который позволяет ПО **понимать и предвосхищать потребности человека**.

- **Естественный язык вместо кода:** LLM позволяют оператору отдавать команды на естественном языке, снижая порог входа и когнитивную нагрузку.
- **Компьютерное зрение для безопасности:** нейросети анализируют позу и усталость оператора в реальном времени и могут скорректировать скорость робота или подать сигнал тревоги.
- **Когнитивная эргономика:** разработчик проектирует интерфейсы, которые не перегружают оператора информацией, а дают интуитивные подсказки и прогнозы.
- **Адаптация в реальном времени:** агенты динамически обновляют рабочие сценарии и взаимодействуют с пользователями через диалоговый интерфейс, обеспечивая немедленную обратную связь и адаптивное управление.

Что это значит для разработчика?

Сдвиг к Индустрии 5.0 меняет **саму суть профессии**:

Таблица 5: Изменение роли разработчика

Раньше (3.0 / 4.0)	Сейчас (5.0)
Писать код, который выполняет задачу	Проектировать системы, которые помогают человеку
Оптимизировать алгоритмы	Оптимизировать взаимодействие человека и машины
Работать с данными	Работать с контекстом и намерениями пользователя
Следовать спецификации	Адаптироваться к изменяющимся потребностям оператора

Как отмечается в исследованиях [8], Индустрия 5.0 — это не революция, а эволюция. Она не заменяет Индустрию 4.0, а дополняет её, фокусируясь на трёх столпах: человеко-центричность, устойчивость и жизнестойкость. Нейросети здесь — не просто «ещё один инструмент» в арсенале разработчика. Это **ключевой энблер**, который позволяет ПО «видеть», «слышать» и «понимать» человека, делая технологии по-настоящему человеко-центричными.

Организация труда и инструменты в эпоху 5.0

Сдвиг в организации команд

- От иерархических структур к **кросс-функциональным, автономным командам**.
- От локальной работы к **глобально распределённым командам** с асинхронными коммуникациями.
- От «инженеров-кодеров» к **инженерам-оркестраторам**, управляющим AI-агентами.

Инструментальная поддержка новых ролей

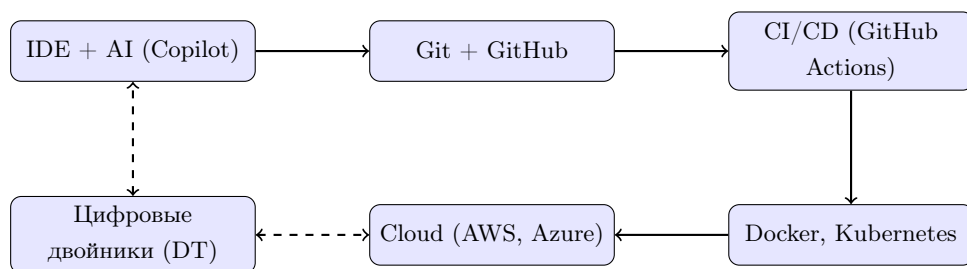


Рис. 3: Современный инструментальный стек с учётом цифровых двойников

Уровни инструментальной поддержки

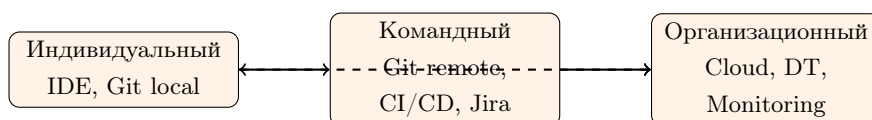


Рис. 4: Уровни инструментальной поддержки в распределённой разработке

Влияние цифровых двойников и эластичных систем на сокращение времени проекта

Факторы ускорения

- **Параллелизация работы** — благодаря Git и распределённым командам.
- **Автоматизация тестирования** — CI/CD выявляет ошибки мгновенно.
- **Унификация окружений** — Docker устраняет проблемы «works on my machine».
- **Прогнозирование сбоев** — цифровые двойники позволяют предвидеть проблемы до их появления.
- **Моделирование сценариев** — двойники позволяют тестировать изменения без риска для продакшена.

Совокупный эффект

По данным исследований, внедрение цифровых двойников и эластичных архитектур может сократить **time-to-market** на 30–50%, снизить количество критических инцидентов на 60% и повысить удовлетворённость пользователей за счёт лучшей адаптации к их потребностям.

Заключение и связь с курсом

Мы рассмотрели эволюцию технологий разработки ПО — от первых машинных кодов до эластичных экосистем Индустрии 5.0. Ввели ключевые определения: технология, инструментальное средство, парадигма, модель ЖЦ. Проанализировали движущие силы сдвига парадигм — от внешних экономических и социальных требований до внутренних законов эволюции ПО (законы Лемана [4, 5]) и катализирующей роли новых абстракций и ИИ. Особое внимание уделили цифровым двойникам как новой парадигме [7], их математическим основам и влиянию на все этапы ЖЦ. Показали, как Индустрия 5.0 [8] меняет требования к ПО и организации труда.

В следующих лекциях мы углубимся в конкретные аспекты:

- Лекция 2 — детальный разбор моделей ЖЦ.
- Лекция 3 — гибкие методологии (Agile, Scrum, Kanban, XP).
- Лекции 4–6 — требования, проектирование (структурное и ООП).
- Лекция 7 — UI/UX проектирование с учётом человеко-центричности.
- Лекции 8–9 — современные инструменты, CI/CD, DevOps, MLOps.
- Лекция 10 — цифровые двойники в инженерии ПО и роль ИИ.

Список литературы

- [1] Соммервилл И. Инженерия программного обеспечения, 10-е изд. — М.: Вильямс, 2016.
- [2] Брукс Ф. Мифический человеко-месяц. — СПб.: Символ-Плюс, 2000.
- [3] Мартин Р. Чистая архитектура. — СПб.: Питер, 2018.
- [4] Lehman M. M. Programs, Life Cycles, and Laws of Software Evolution // Proceedings of the IEEE. 1980. Vol. 68. No. 9. P. 1060–1076.

- [5] Lehman M. M. Laws of software evolution revisited // Lecture Notes in Computer Science. 1996. Vol. 1149. P. 108–124.
- [6] Lehman M. M., Belady L. A. Program Evolution-Processes of Software Change. Academic Press, 1985.
- [7] Grieves M., Vickers J. Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems // Springer, 2017.
- [8] European Commission. Industry 5.0 — Towards a sustainable, human-centric and resilient European industry. 2021.
- [9] Kuhn T. The Structure of Scientific Revolutions. University of Chicago Press, 1962.
- [10] Андреев А.В. и др. Цифровые двойники: синтез теории сложности и искусственного интеллекта // Известия вузов. ПНД, 2026.